2021-2022

UE Code : HAE916E

ARTIFICIAL INTELLIGENCE

Realized by :

Yacine Mehadji

Fields:

Microelectronics

Master's Degree:

Integrated and Embedded Electronics Systems

Supervisor:

Neha Baranwal

Artificial intelligence (**AI**) is intelligence demonstrated by machines, as opposed to **natural intelligence** displayed by animals including humans. **Machine learning** (**ML**) is the study of computer algorithms that can improve automatically through experience and by the use of data. It is seen as a part of artificial intelligence. **Deep learning** is part of a broader family of machine learning methods based on artificial neural networks with representation learning. Learning can be supervised, semi supervised or unsupervised.
This research shows three models of Machine Learning PCA, SVM and Artificial Neural Networks.

1 **INTRODUCTION** **3**
2 **PRACTICAL WORK** **3**
2.1 PRINCIPAL COMPONENT ANALYSIS 3
2.1.1 Applying PCA to the flower dataset *3*
2.1.2 Conclusion: *5*
2.2 SUPPORT VECTOR MACHINE 5
2.3 NEURAL NETWORK 6
2.3.1 Programming the Network *7*
2.3.2 Training and testing the network on three samples. *7*
2.3.3 Conclusion: *8*
3 **PCA_SVM PYTHON CODE** **10**
4 **NEURAL NETWORKS PYTHON CODE** **14**

1 Introduction

The objective of these practical works is to get a grasp of Principal Component Analysis (PCA), Support Vector Machines (SVM) and Neural Networks with Python.
First, the goal is to classify botanical data associated to flower species,using supervised machine learning. The method is supervised, in the sense that you have access to flowerTrain.data, a dataset quantifying the morphologic variation of flowers along three species: interior, versicolor and convoluta. The dataset (which you can read with any text editor) consists of 48 samples for each of the three species. Four features have been measured from each sample: length and width of the sepals, and length and width of the petals. Then with Python getting to know about Neural Networks, starting by programming the network after that Training and Testing the Network on three samples and on a large Dataset.

2 Practical Work

2.1 Principal Component Analysis

Definition:
Large datasets are increasingly common and are often difficult to interpret. Principal component analysis (PCA) is a technique for reducing the dimensionality of such datasets, increasing interpretability but at the same time minimizing information loss. It does so by creating new uncorrelated variables that successively maximize variance. Finding such new variables, the principal components.

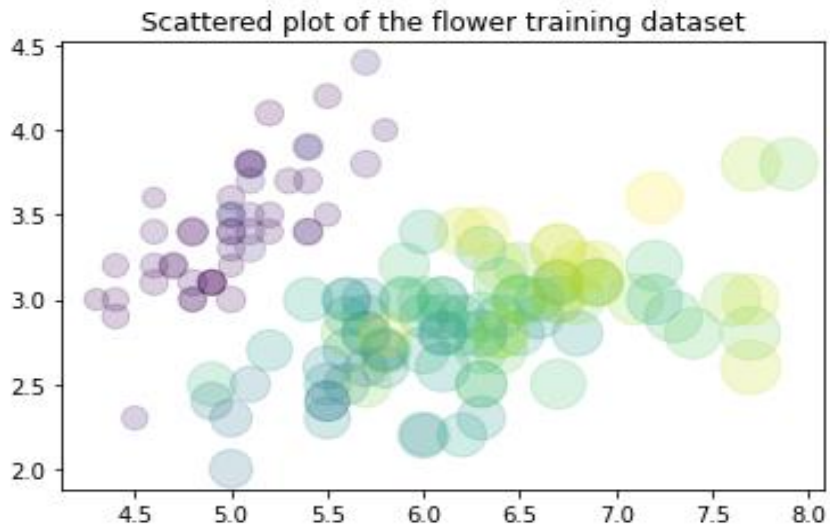
2-1-1 Practicing PCA on two examples

$$X1 = \begin{bmatrix} 1 & 5 & 3 & 3 \\ 4 & 4 & 3 & 5 \end{bmatrix}, \qquad X2 = \begin{bmatrix} 1.268 & 4.732 & 3.5 & 2.5 \\ 3 & 5 & 3.134 & 4.866 \end{bmatrix}$$

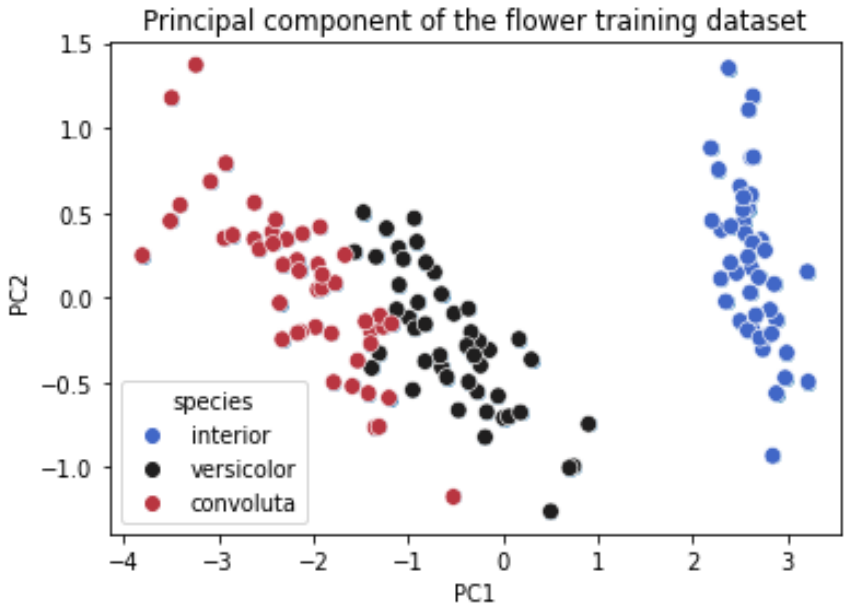
Using PCA into these examples, we have been able to reduce the data from 2 dim to 1 dim, to implement the six steps of PCA.

2.1.1 Applying PCA to the flower dataset

The plot of the flower dataset before applying PCA is :



We may now apply PCA(X , reducedDim) to the flower dataset. Since the dataset has dimension four, plotting the eigenvectors is not relevant here.
The two-Dimensional plot of the samples is :



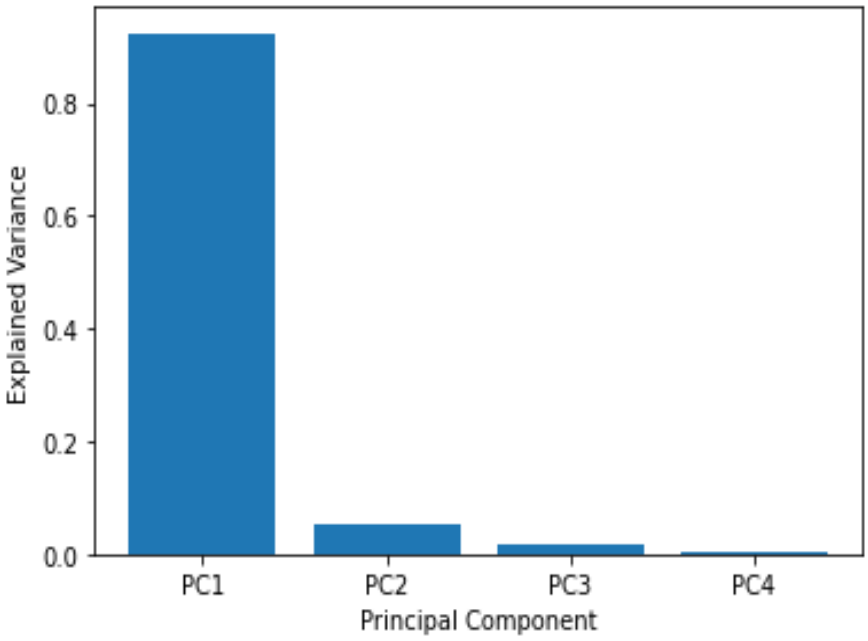
Correlations coefficients of original dataset is : -0.11394505934445667 0.17387516288098412.

There is a strong and statistically significant correlation between SEPAL LENGTH and SEPAL WIDTH.

The PCA makes it possible to decorrelate these variables.

Correlations coefficients of PC1 and PC2 is : 2.0296264668928643e-16 0.9999999999999957

There is no linear correlation between the two principal components, PCA transforms the correlated variables of a dataset into a new set of uncorrelated variables.



PCA model attribute to find the amount of variance explained by each component (PC1 and PC2), The maximum of information is stored in PC1.

2.1.2 Conclusion:

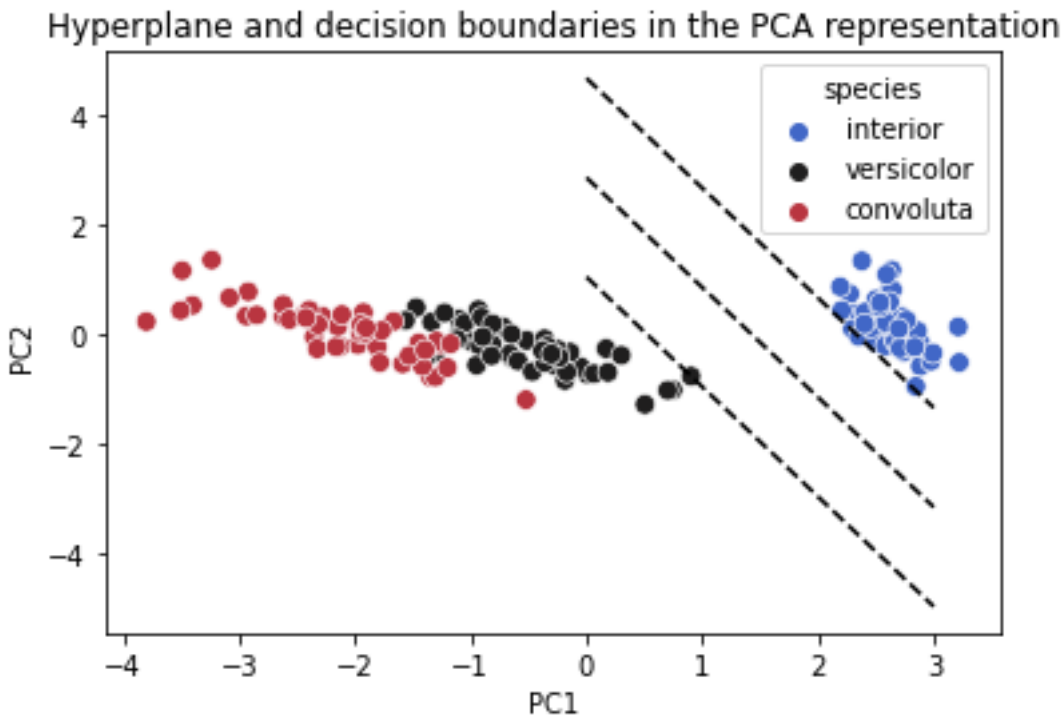
We could see throught this first part of the TP, the method of principal component analysis which consists in transfoming variables related to each other (called correlated) into new variables are named principal component analysis.
We have verified it by calculating the correlation coefficient between SEPAL LENGTH and SEPAL WIDTH there is a strong correlation between these two samples their correlation coefficient is close. By Using the PCA method, we have been able to decorrelate these samples, and their correlation coefficient are not close.

In plots of the two principal components, clusters of samples of the three species are much clearer than in the original 4-D space. In particular, there is a clear separation between interior and the two other species. In the next section, you will design a linear classifier (support vector machine) for determining whether a sample corresponds to an interior flower or not.

2.2 Support Vector Machine

Definition:

Support Vector Machine is supervised learning models with associated learning algorithms that analyze data for classification and regression analysis.
SVM offers very high accuracy compared to other classifiers such as logistic regression and decision trees. It is known for its kernel trick to handle nonl inear input spaces. It is used in a variety of applications such as face detection, intrusion detection and handwriting recognition. SVM is an exciting algorithm, it separates data points using a hyperplane with the largest margin.

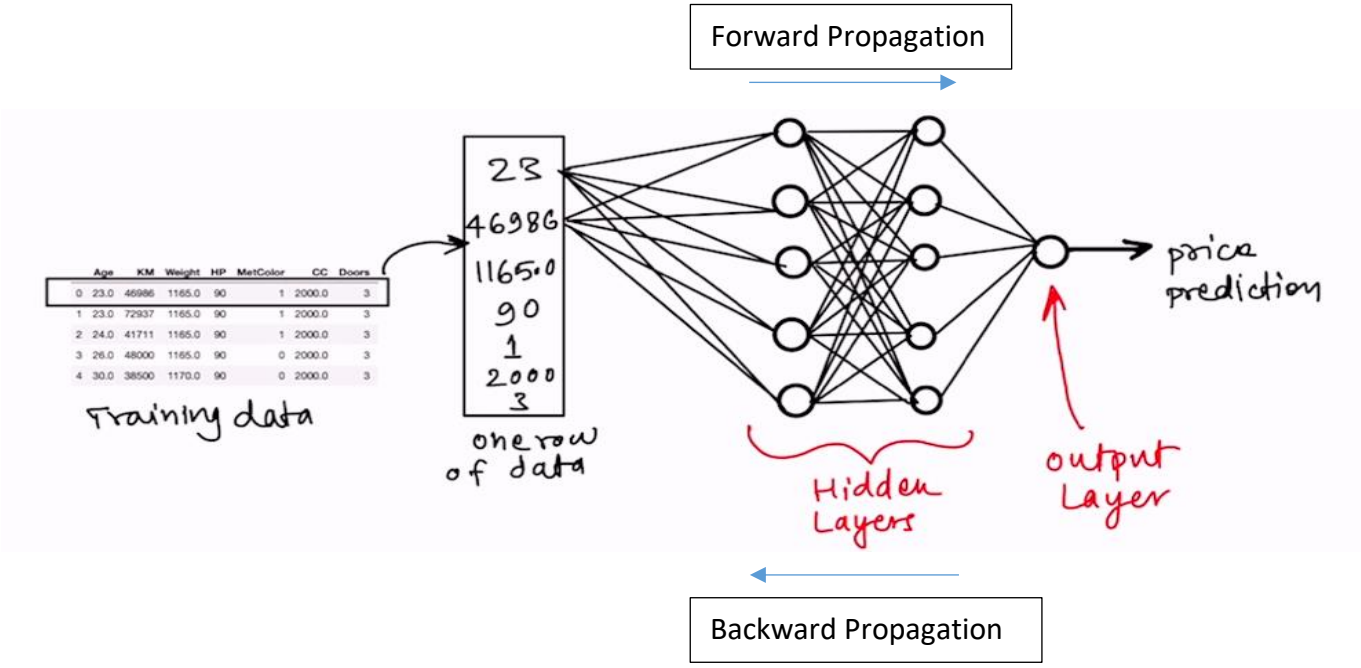


[PCA_SVM Python Code : here]

2.3 Neural Network

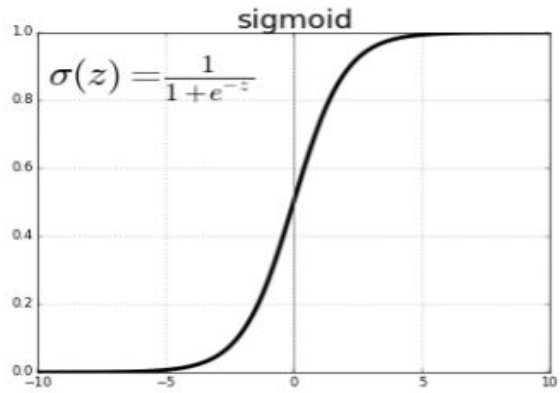
For developing and training Artificial Networks we repeat the following 04 steps :

- Forward Propagation: We circulate the data from the 1st layer to the last one in order to produce an output y .
- Cost Function: calculate the error between this output and the reference output (y output and desired ytrue output).
- Backward Propagation: We measure how this cost function varies with respect to each layer of our model starting from the last one and going back to the first.
- Gradient Descent: correct each parameter of the model thanks to the gradient descent algorithm before looping back to the 1st step of forward propagation.



Architecture of Neural Network used in this case

the output of our model is a Sigmoid function that can be seen as a probability always between 0 and 1.



Activation Function of Neural Networks

2.3.1 Programming the Network

Using Python, we design the network shown in the figure above, we start by two labeled samples each samples contains input x with corresponding target output t :

$$X1=\begin{pmatrix}2\\1\end{pmatrix} \qquad t1=\begin{pmatrix}1\\0\end{pmatrix} \qquad X2=\begin{pmatrix}-1\\3\end{pmatrix} \qquad t2=\begin{pmatrix}0\\1\end{pmatrix}$$

We initialize the network with the following weights:

$$W = [2 \quad -3 \quad -3 \quad 4 \quad 1 \quad -1 \quad 0.25 \quad 2]^T$$

We design a function for the forward pass, and we obtain :

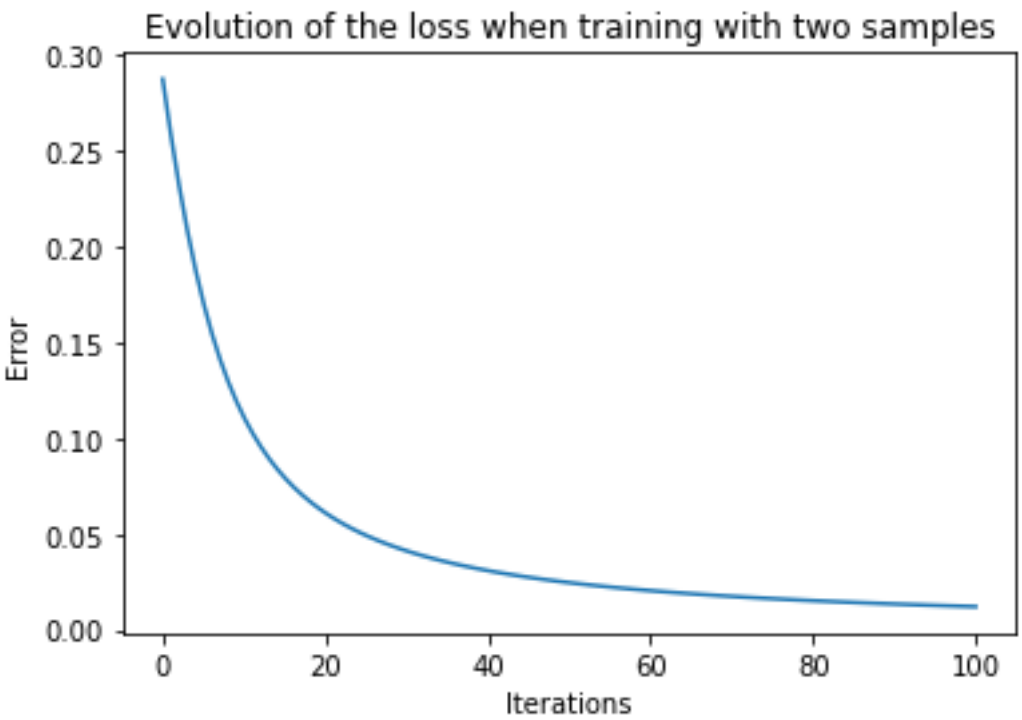
Loss 1 (L1) = 0.245
Loss 2 (L2) = 0.33

We design a function for the backwardpropagation of the network, we obtain :

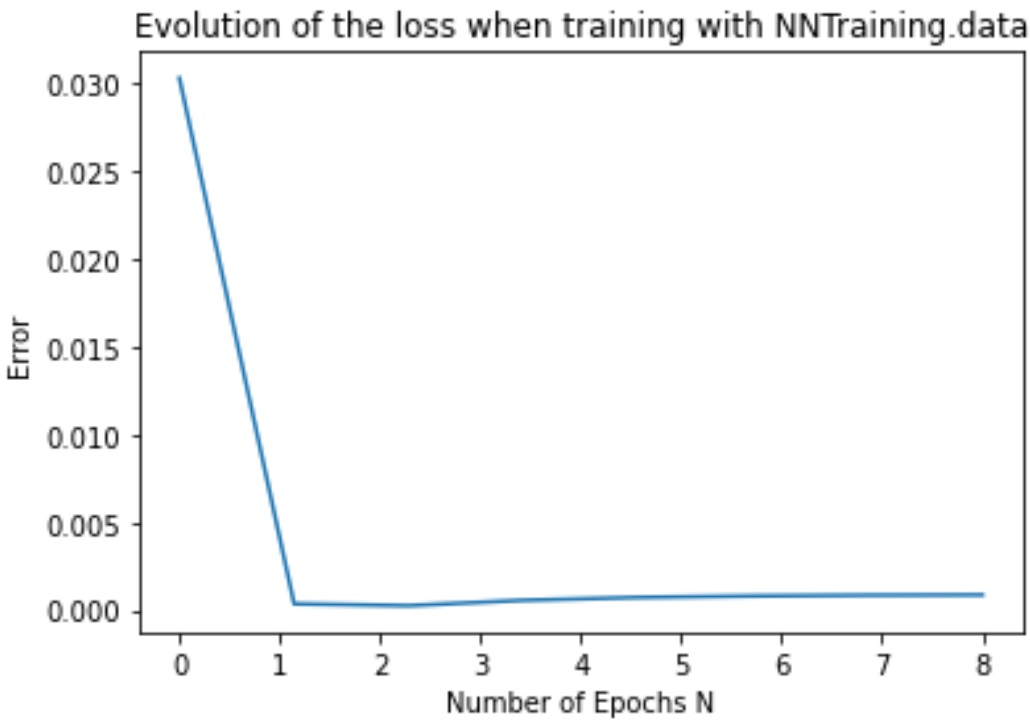
$$\text{Grad1} = [-0.12 \quad 0.07 \quad -0.06 \quad 0.03 \quad -0.10 \quad 0.13 \quad -0.02 \quad 0.02]^T$$
$$\text{Grad2} = [\sim 0 \quad \sim 0 \quad \sim 0 \quad \sim 0 \quad \sim 0 \quad \sim 0 \quad 0.28 \quad -0.025]^T$$

2.3.2 Training and testing the network on three samples.

With the same dataset, run the network (forward and back) to train it until the loss function has diminished enough. Since the dataset consists in only two samples, we use a single batch coinciding with the whole dataset.



Interpretation: We obtain our learning curve, that means that the evolution of the errors made by the model as it learns, we see that the errors decreases and that the cost function "LOSS" converges to a limit plateau value.



[Neural Networks Python Code : here]

2.3.3 Conclusion:

In this practical work, we have been able to build a Neural Network using Numpy in Pyhton, then we learned what deep learning is and what differentiates it from machine learning, how to represent vectors with numpy, what activation functions are and why they're used inside a Neural Networks, and we designed forward pass and backpropagation algorithm is and how it works, finally how to train a Neural Network and make predictions.

3 PCA_SVM Python Code

```
# -*- coding: utf-8 -*-
"""
@author: Yacine Mehadji
"""

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.svm import SVC
import seaborn as sb
from sklearn.model_selection import train_test_split
from scipy.stats import pearsonr
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn import metrics

"""--------------------Practicing PCA on two examples----------------------"""

#data=np.matrix([[1,5,3,3],[4,4,3,5]])
# moyennedata=np.mean(data)
# print(data)
# plt.figure()
# plt.scatter(data[:,0],data[:,1])


#B=np.array([[1.268,4.732,3.5,2.5],[3,5,3.134,4.866]])
# moyenneB=np.mean(B)
#plt.figure()
#plt.scatter(B[:,0],B[:,1])
#print(B)

print('Using PCA into these examples, we have been able to reduce the data from 2 dim to 1 dim, to implement the six steps of PCA')

"""------------------Applying PCA to the flower dataset--------------------"""

data = pd.read_csv("flowerTrain.data", names=['sepal length','sepal width','petal length','petal width','species'])
x = data.iloc [:,0:4]
plt.figure()
plt.scatter(data.iloc [:,0], data.iloc [:,1], alpha=0.2, s=100*data.iloc [:,2], c=100*data.iloc [:,3])
plt.title('Scattered plot of the flower training dataset')

def PCA(x , reducedDim):
    mean = np.mean(x, axis= 0) #Compute the mean of all measures
    print(mean)
    mean_data = x - mean      #Shift matrix by this mean
    print(mean_data, "\n")
    #plt.figure()
    #plt.scatter(mean_data.iloc [:,0], mean_data.iloc [:,1], alpha=0.2, s=100*data.iloc [:,2], c=100*data.iloc [:,3])
    #plt.title('Mean Data')
```

```
cov = np.cov(mean_data.astype(float), rowvar=False) #Build the dataset covariance matrix
print("Covariance matrix ", cov.shape, "\n")
print(cov)

eig_vals, eig_vecs = np.linalg.eigh(cov) #Using Singular Value Decomposition, extract eigenvalues and normalized eigenvectors
idx = eig_vals.argsort()[::-1] #choose the p largest eigenvalues and reduce U to the corresponding p eigenvectors (columns)
eig_vals = eig_vals[idx]
eig_vecs = eig_vecs[:,idx]
print("\nEigen vectors are :\n ", eig_vecs)
print("\nEigen values are : ", eig_vals, "\n")


Up = eig_vecs.T[:][:2].transpose()
S = np.dot(Up.transpose(),mean_data.transpose()).transpose() #project each sample to reduce its dimensions
print("Principal Component Size", S.shape, "\n")
principal_df = pd.DataFrame(S , columns = ['PC1','PC2'])
principal_df = pd.concat([principal_df , data.loc[:,['species']]] , axis = 1)

plt.figure()
plt.scatter(principal_df['PC1'], principal_df['PC2'])
sb.scatterplot(data = principal_df , x = 'PC1',y = 'PC2' , hue = 'species' , s = 60 , palette= 'icefire')
plt.title('Principal component of the flower training dataset')

#Correlation Coefficient
coef_cor_data, pvalue_data = pearsonr(data.iloc[:,0], data.iloc[:,1])
print('correlation coefficient of original dataset : ', coef_cor_data, pvalue_data)
print('\nThere is a strong and statistically significant correlation between SEPAL LENGTH and SEPAL WIDTH')
print('\nThe PCA makes it possible to decorrelate these variables')

PC1 = S[:,0] #The first principal component
PC2 = S[:,1] #The second principal component

coef_cor_PC, pvalue_PC = pearsonr(PC1,PC2)
print('\ncorrelation coefficient of PC1 and PC2 : \n', coef_cor_PC, pvalue_PC)
print('\nThere is no linear correlation between the two principal components, PCA transforms the correlated variables of a dataset into a new set of uncorrelated variables')

#PCA model attribute to find the amount of variance explained by each component (PC1 and PC2)
sum_eig_vals = np.sum(eig_vals)
explained_variance = eig_vals/ sum_eig_vals
print('\nExplained Variance for each component : ', explained_variance)

cumulative_variance = np.cumsum(explained_variance)
print('\nCumulative Variance : ', cumulative_variance)

plt.figure()
plt.bar(range(4),explained_variance)
plt.xlabel('Principal Component')
```

```
    plt.ylabel('Explained Variance')
    plt.xticks(range(4),['PC1','PC2','PC3','PC4'])
    plt.show()

    return S

PCA(x,2)

print('\nConclusion : \n')
print('We could see throught this first part of the TP, the method of principal component analysis which consists in transfoming variables related to each other (called correlated) into new variables are named principal component analysis.')
print('We have verified it by calculating the correlation coefficient between SEPAL LENGTH and SEPAL WIDTH there is a strong correlation between these two samples their correlation coefficient are close.')
print('By Using the PCA method, we have been able to decorrelate these samples, and their correlation coeffcient are not close')



"""-------------------------------Datasets----------------------------------"""
S = PCA(x,2)
principal_df = pd.DataFrame(S , columns = ['PC1','PC2'])
principal_df = pd.concat([principal_df , data.loc[:,['species']]] , axis = 1)
plt.figure()
plt.scatter(principal_df['PC1'], principal_df['PC2'])
sb.scatterplot(data = principal_df , x = 'PC1',y = 'PC2' , hue = 'species' , s = 60 , palette= 'icefire')

"""--------------------------Support Vector Machine-------------------------"""

y = data.loc[:,['species']].values

new_y = np.zeros(144, dtype=int)
for i in range (len(principal_df['species'])):
    if principal_df.loc[i,'species']=='interior':
        new_y[i]=0
    else:
        new_y[i]=1

svm = SVC(kernel="linear", C=1E10)
svm.fit(S, new_y)

w=svm.coef_[0]
a = -w[0] / w[1]
xx = np.linspace(0, 3)
yy = a* xx - (svm.intercept_[0] / w[1])

marg = 1/np.sqrt(np.sum(svm.coef_**2))
yy_min=yy-np.sqrt(1+a**2)*marg
yy_max=yy+np.sqrt(1+a**2)*marg
plt.figure()
plt.plot(xx,yy,"k--")
plt.plot(xx,yy_min,"k--")
```

```
plt.plot(xx,yy_max,"k--")
sb.scatterplot(data = principal_df , x = 'PC1',y = 'PC2' , hue = 'species' , s = 60 , palette= 'icefire')
plt.title('Hyperplane and decision boundaries in the PCA representation')

#Evaluating the model
pred_y = svm.predict(S)
confusion_matrix(new_y,pred_y)

print("Accuracy : ",metrics.accuracy_score(new_y,pred_y))
```

4 Neural Networks Python Code

```
# -*- coding: utf-8 -*-
"""
Created on Wed Nov 24 11:48:11 2021

@author: Yacine Mehadji
"""

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def sigmoid(z):
    return 1/(1+np.exp(-z))

def forwardpass(x,t,w):
    x_1=x[0]
    x_2=x[1]

    t_1=t[0]
    t_2=t[1]

    w_0 = w[0]
    w_1 = w[1]
    w_2 = w[2]
    w_3 = w[3]
    w_4 = w[4]
    w_5 = w[5]
    w_6 = w[6]
    w_7 = w[7]

    #forwardpass
    h_1in = w_0* x_1 + w_2 * x_2
    h_2in = w_1* x_1 + w_3 * x_2

    h_1out = sigmoid(h_1in)
    h_2out = sigmoid(h_2in)

    y_1in = w_4* h_1out + w_6 * h_2out
    y_2in = w_5* h_1out + w_7 * h_2out

    y_1out = sigmoid(y_1in)
    y_2out = sigmoid(y_2in)


    #The Loss
    L = (y_1out - t_1)**2 + (y_2out - t_2)**2

    #backpropagation
    delta_y1out = 2*(y_1out-t_1)
    delta_y2out = 2*(y_2out-t_2)

    delta_y1in = delta_y1out * sigmoid(y_1in)*(1-sigmoid(y_1in))
```

```
    delta_y2in = delta_y2out * sigmoid(y_2in)*(1-sigmoid(y_2in))

    delta_w4 = delta_y1in * h_1out
    delta_w5 = delta_y2in * h_1out

    delta_w6 = delta_y1in * h_2out
    delta_w7 = delta_y2in * h_2out

    delta_h1out = delta_y1in * w_4 + delta_y2in * w_5
    delta_h2out = delta_y1in * w_6 + delta_y2in * w_7

    delta_h1in = delta_h1out * sigmoid(h_1in)*(1-sigmoid(h_1in))
    delta_h2in = delta_h2out * sigmoid(h_2in)*(1-sigmoid(h_2in))

    delta_w0 = delta_h1in * x_1
    delta_w1 = delta_h2in * x_1

    delta_w2 = delta_h1in * x_2
    delta_w3 = delta_h2in * x_2
    grd = np.array([delta_w0,delta_w1,delta_w2,delta_w3,delta_w4,delta_w5,delta_w6,delta_w7])
    return L, grd.T

#########################################
#2-Programming the Network
#########################################

x1 = np.array([[2.],[1]])
t1 = np.array([[1.],[0]])

x2 = np.array([[-1.],[3]])
t2 = np.array([[0.],[1]])

w = np.array([2.,-3,-3,4,1,-1,0.25,2])

L1,grd1 = forwardpass(x1, t1, w)
L2,grd2 = forwardpass(x2, t2, w)

print('Value of Loss 1 L1 is : ', L1)
print('Value of Loss 1 L2 is : ', L2)

print('Gradient 1 is : ', grd1)
print('Gradient 2 is : ', grd2)

##############################################
#3.1-Training and Testing on three samples
##############################################

i = 0
N = 100
wi = np.zeros((N+1,8))
L = np.zeros((N,1))
p = 1
tol = 10**-3
```

```
wi[0,:] = w
for i in range(0,N):
    L1,grd1 = forwardpass(x1, t1, wi[i,:])
    L2,grd2 = forwardpass(x2, t2, wi[i,:])
    grd = (grd1+grd2)/2
    wi[i+1,:] = wi[i,:] - p * grd
    L[i]=(L1+L2)/2
t=np.linspace(0,100,100)
plt.xlabel('Iterations')
plt.ylabel('Error')
plt.title('Evolution of the loss when training with two samples')
plt.plot(t,L)
print('\n Interpretation of curve 1 : We obtain our learning curve, that means that the evolution of the errors made by the model as it learns, we see that the errors decreases and that the cost function "LOSS" converges to a limit plateau value. \n')

x3 = np.array([[1.],[4]])
t3 = np.array([[1.],[0]])
L3,grd3 = forwardpass(x3, t3, wi[N,:])

##############################################
#3.2-Training and Testing on a large Dataset
##############################################

data=pd.read_csv('NNTraining.data')
X_1 = data.iloc[:,0]
X_2 = data.iloc[:,1]
T_1 = data.iloc[:,2]
T_2 = data.iloc[:,3]

i = 0
N = 8
p = 6
tol = 10**-3
W = np.zeros((2,8))
W[0,:] = w
m=35
[l,c] = np.shape(data)
L = np.zeros((N,0))
Lm = np.zeros((N,1))
ggdr = np.zeros((l,8))
for i in range(0,N):

    for k in range (0,np.int(200/m)):
        sup = np.int((k+1)*200/m)
        infe = np.int(k*200/m)
        for j in range(infe,sup):
            L,ggdr[j] = forwardpass([X_1[j],X_2[j]], [T_1[j],T_2[j]], W[0,:])
        grd[0,0] = np.sum(ggdr[0:m,0])/m
        grd[0,1] = np.sum(ggdr[0:m,1])/m
        grd[0,2] = np.sum(ggdr[0:m,2])/m
        grd[0,3] = np.sum(ggdr[0:m,3])/m
        grd[0,4] = np.sum(ggdr[0:m,4])/m
```

```
    grd[0,5] = np.sum(ggdr[0:m,5])/m
    grd[0,6] = np.sum(ggdr[0:m,6])/m
    grd[0,7] = np.sum(ggdr[0:m,7])/m

    W[1,:] = W[0,:] - p * grd
    W[0,:] = W[1,:]

    Lm[i]= L
plt.figure()
t=np.linspace(0,N,N)
plt.plot(t,Lm)
plt.xlabel('Number of Epochs N')
plt.ylabel('Error')
plt.title('Evolution of the loss when training with NNTraining.data')
plt.show()

#########################################

# data1=pd.read_csv('NNTest.data')
# X_1_T = data.iloc[:,0]
# X_2_T = data.iloc[:,1]
# T_1_T = data.iloc[:,2]
# T_2_T = data.iloc[:,3]

# i1 = 0
# N1 = 100
# p1 = 200
# tol1 = 10**-3
# W1 = np.zeros((2,8))
# W1[0,:] = w
# m1=100
# [l1,c1] = np.shape(data1)
# L1 = np.zeros((N1,0))
# Lm1 = np.zeros((N1,1))
# ggdr1 = np.zeros((l1,8))
# for i1 in range(0,N1):

#    for k1 in range (0,np.int(200/m)):
#       sup1 = np.int((k1+1)*200/m1)
#       infe1 = np.int(k1*200/m1)
#       for j1 in range(infe1,sup1):
#          L1,ggdr1[j1] = forwardpass([X_1_T[j],X_2_T[j]], [T_1_T[j],T_2_T[j]], W1[0,:])
#       grd1[0,0] = np.sum(ggdr1[0:m,0])/m1
#       grd1[0,1] = np.sum(ggdr1[0:m,1])/m1
#       grd1[0,2] = np.sum(ggdr1[0:m,2])/m1
#       grd1[0,3] = np.sum(ggdr1[0:m,3])/m1
#       grd1[0,4] = np.sum(ggdr1[0:m,4])/m1
#       grd1[0,5] = np.sum(ggdr1[0:m,5])/m1
#       grd1[0,6] = np.sum(ggdr1[0:m,6])/m1
#       grd1[0,7] = np.sum(ggdr1[0:m,7])/m1
#                                                                                    # grd =
np.sum(np.sum(GRD[0*200/4:1*200/4])/(200/4)+np.sum(GRD[1*200/4:2*200/4])/(200/4)+n
p.sum(GRD[2*200/4:3*200/4])/(200/4)+np.sum(GRD[3*200/4:4*200/4])/(200/4)) /4
```

```
#       W1[1,:] = W1[0,:] - p1 * grd1
#       W1[0,:] = W1[1,:]

#                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
# np.sum(np.sum(GRD[0*200/4:1*200/4])/(200/4)+np.sum(GRD[1*200/4:2*200/4])/(200/4)+np.sum(GRD[2*200/4:3*200/4])/(200/4)+np.sum(GRD[3*200/4:4*200/4])/(200/4)) /4
#     Lm1[i1]= L1
# plt.figure()
# t1=np.linspace(0,N1,N1)
# plt.plot(t1,Lm1)
# plt.show()
```